firma_tap

Documentación del Proyecto

Descripción General

firma_tap es una aplicación móvil Flutter para captura y gestión de firmas digitales de waybills (albaranes), diseñada para operar en entornos con conectividad intermitente y sincronizarse con el servidor bridge-firma.

Arquitectura

```
┌──────────────────────────────────────────────────────┐
│                        CAPAS                         │
│                                                      │
│ 1. Presentation Layer     UI widgets, pages, navigation  │
│ 2. Business Logic Layer   Servicios, repositorios, models │
│ 3. Data Layer             Hive local DB + SecureStorage  │
│ 4. Integration Layer      REST API bridge-firma + Workmanager │
└──────────────────────────────────────────────────────┘
```

Versión actual

v1.0 - `feature/bridge-integration`

Características Principales

Categoría	Funcionalidad
Base de Datos Local	Hive (offline-first, persiste entre reinicios)
Autenticación	`local_auth` + `flutter_secure_storage`
Cola Offline	`Workmanager` tarea cada 15 min + `QueueService` connectivity listener
Verificación de Hash	SHA256 `HistoryService.computeHash` (patrón bridge-firma)
Historial	`HistorialEntry` en Hive `historial` box (typeId 11)
Empresa Management	`EmpresaService` + catálogo remoto + storage local
Biometría	`LocalAuthentication` + `FlutterSecureStorage`
Notificaciones	`firebase_messaging` + background handler
Gestión de Colas	`QueueService` + retry on connectivity change
Generación PDF	`pdfx` package para leer PDFs del servidor
Reconocimiento de Texto	`google_mlkit_text_recognition`
Geolocalización	`geolocator` con ubicación en firma

Estructura del Proyecto

```
firma_tap/
├── lib/
│   ├── main.dart                # Punto de entrada + Workmanager init
│   ├── core/
│   │   ├── auth/                # biometric_service.dart, auth_storage.dart
│   │   ├── config.dart
│   │   ├── constants/           # api_constants.dart, env.dart
│   │   ├── offline/             # queue_service.dart
│   │   ├── push/                # push_service.dart
│   │   ├── storage/             # history_service.dart, auth_storage.dart
│   │   └── theme/               # app_theme.dart
│   ├── features/
│   │   ├── pages/               # 18 screens (login, home, documents, etc.)
│   │   ├── models/              # firma_dto, albaran_dto, empresa_dto, etc.
│   │   ├── services/            # firma_service, albaran_service, empresa_service
│   │   └── repositories/        # albaran_repository
│   └── generated/               # Generated code
├── test/                        # Tests unitarios
├── pubspec.yaml                 # Dependencias Flutter
└── assets/                      # Imágenes, TFLite models, certificados
```

Dependencias Principales

Categoría	Paquetes
Flutter SDK	`flutter`, `flutter_lints`
State Management	`flutter_animate`
Local DB	`hive`, `hive_flutter`
Auth	`local_auth`, `shared_preferences`, `flutter_secure_storage`
Network	`http`, `connectivity_plus`
Firebase	`firebase_core`, `firebase_messaging`
ML/OCR	`tflite_flutter`, `google_mlkit_text_recognition`
PDF	`pdfx`
Location	`geolocator`
Scheduling	`workmanager`
Sentry	`sentry_flutter`

Flujo de Trabajo Completo

1. Login

```
Usuario → Ingresa ID + Selecciona Empresa
          ↓
AuthStorage.saveSession() → guarda en SharedPreferences
          ↓
BiometricService.preguntarActivarBiometría() → prompt "¿Entrar más rápido?"
          ↓
if user accepts → BiometricService.authenticateDetailed()
          ↓
if ok → BiometricService.saveCredentials() → guarda en SecureStorage
          ↓
Navigator.pushReplacement → HomePage(usuarioId, empresa)
```

2. Firmar Documento

```
HomePage → Navigator → SelectUserPage → elegir usuario/empresa
           ↓
Navigator → DocumentosPage → lista de albaranes pendientes
           ↓
Usuario selecciona documento → Navigator → FirmaDocumentoPage
           ↓
Canvas signature → toPngBytes() → base64Encode
           ↓
POST /guardar_firma (bridge-firma)
           ↓
Guardar PNG + hash SHA256 → BD actualizar estado FIRMADO
           ↓
Inserta en historial → Navigator.pop hasta home
```

3. Cola Offline y Retry

```
Si sin internet al firmar:
   ↓
firma_tap usa Hive 'cola_firmas' box → store FirmaPendiente(local)
   ↓
Workmanager task every 15 min → reintenta pendientes
   ↓
Si aún falla → incrementa intentos counter
   ↓
Si éxito → mueve historial, limpia cola
```

4. Historial y Consultas

```
HomePage →_navigate → HistorialPage
   ↓
Lista de firma_historial entries
   ↓
Cada entrada tiene: num_doc, usuario_id, empresa_codigo, fecha_firma,
hash SHA256, observaciones
   ↓
GET /api/historial (bridge-firma) → sincronizar con servidor
```

Integración con bridge-firma

Endpoints Consumidos

Propósito	Endpoint	Response
Cargar documentos pendientes	`GET /api/pendientes`	Lista de documentos con id, num_doc, etc.
Empresa catalog	`GET /api/empresas`	`[{codigo, descripcion}]`
Guardar firma	`POST /guardar_firma`	`{"status": "success", "firma_hash": "abc123...", "ruta": "firma_1.png"}`
Historial synchronización	`GET /api/historial`	Lista de firmas completadas

Patrón de Sincronización

```
// Al iniciar la app o cada X minutos
Future<void> sincronizarConServidor() async {
  final pending = await http.get(Uri.parse('http://192.168.1.50:5000/api/pendientes'));
  final listaPendientes = pending.json();

  for (final doc in listaPendientes) {
    // Verificar si ya está localmente
    final local = await Hive.box<FirmaPendiente>('cola_firmas').get(doc['num_doc']);

    if (local == null) {
      // Nuevo documento desde servidor - cargar PDF y mostrar canvas
      // ... navegación a FirmaDocumentoPage
    } else if (local.intentos < 3) {
      // Intentar reenvío si tiene intentos pendientes
      await _enviarFirma(
        jpgBytes: local.jpgBytes,
        codigoEmpresa: local.codigoEmpresa,
        tipoDocumento: local.tipoDocumento,
        numero: local.numero,
        usuario: local.usuario,
      );
    }
  }

  // Sincronizar historial
  final historial = await http.get(Uri.parse('http://192.168.1.50:5000/api/historial'));
  // ...merge con Hive box 'historial'
}
```

Mejoras Aplicadas (feature/bridge-integration)

Archivo	Mejoras
`lib/core/auth/auth_storage.dart`	`saveSession()` con flags de biometría, `hasSession()`, `hasMinimalSession()`, `clear()` completo, `saveBiometryFlags()`
`lib/core/auth/biometric_service.dart`	`computeHash()` SHA256 estático, `validarIntegridad()` comparación de hashes, logging de auditoría en `finally` block

Comandos Útiles

```
# Desde el directorio raíz del proyecto Flutter
flutter pub get          # Obtener dependencias
flutter run              # Ejecutar en dispositivo/emulador
flutter build apk        # Compilar APK release
flutter build appbundle  # Compilar app bundle

# Ver logs
flutter logs

# Testear integración con bridge-firma
curl http://192.168.1.50:5000/api/pendientes

# Probar endpoint de firma
curl -X POST http://192.168.1.50:5000/guardar_firma \
  -H "Content-Type: application/json" \
  -d '{"id":1,
       "firma_b64":"
         CAQAAAC1HAwCAAAAC0lEQVR42mNk+A8AAQUQnCk8A4YAAAAASUVORK5CYII=",
       "usuarioId":1,
       "empresaCodigo":"EC"}'
```

Nota: el JSON de ejemplo se ha dividido en varias líneas solo para que quepa en la página; en el comando real va en una sola línea.

Comandos de Despliegue

```
# Compilar para producción
flutter pub get
flutter build apk --release
flutter build appbundle --release

# Ejecutar en modo debug
flutter run

# Obtener dependencias nuevas
flutter pub add <package_name>
```

Licencia y Autoría

- **Proyecto creado y mantenido por:** josearregui
- **Fecha:** Octubre 2026
- **Versión:** firma_tap v1.0 con integración bridge-firma
- **Rama actual:** feature/bridge-integration

Contacto y Soporte

- **Issue Tracker:** Repositorio Git oficial
- **Documentación Adicional:** Ver `lib/core/auth/` y `lib/features/`
- **Soporte Técnico:** Revisar `pubspec.yaml` y `analysis_options.yaml`